

Ingeniería inversa para análisis de software

Conceptos y Herramientas
Enfoques de Ingeniería Inversa

¿Qué es la Ingeniería Inversa?

- Proceso de desensamblar, descompilar o analizar un software sin acceso al código fuente.
- Objetivos:
 - Comprender el funcionamiento interno.
 - Identificar vulnerabilidades y malware.
 - Verificar cumplimiento de estándares/licencias.
 - Recuperar conocimiento perdido.
 - Ejemplo: analizar un ejecutable sin documentación.

Enfoques de Ingeniería Inversa



Estático:

Se examina el binario o código sin ejecutarlo.

Ejemplo:
desensamblado,
análisis de cadenas,
firmas.



Dinámico:

Se estudia el comportamiento durante la ejecución.

Ejemplo: debugging,
monitoreo de API,
sandboxing.



Híbrido:

Combina ambos enfoques para un análisis más completo.

Historia y evolución



Años 80: análisis de virus en MS-DOS

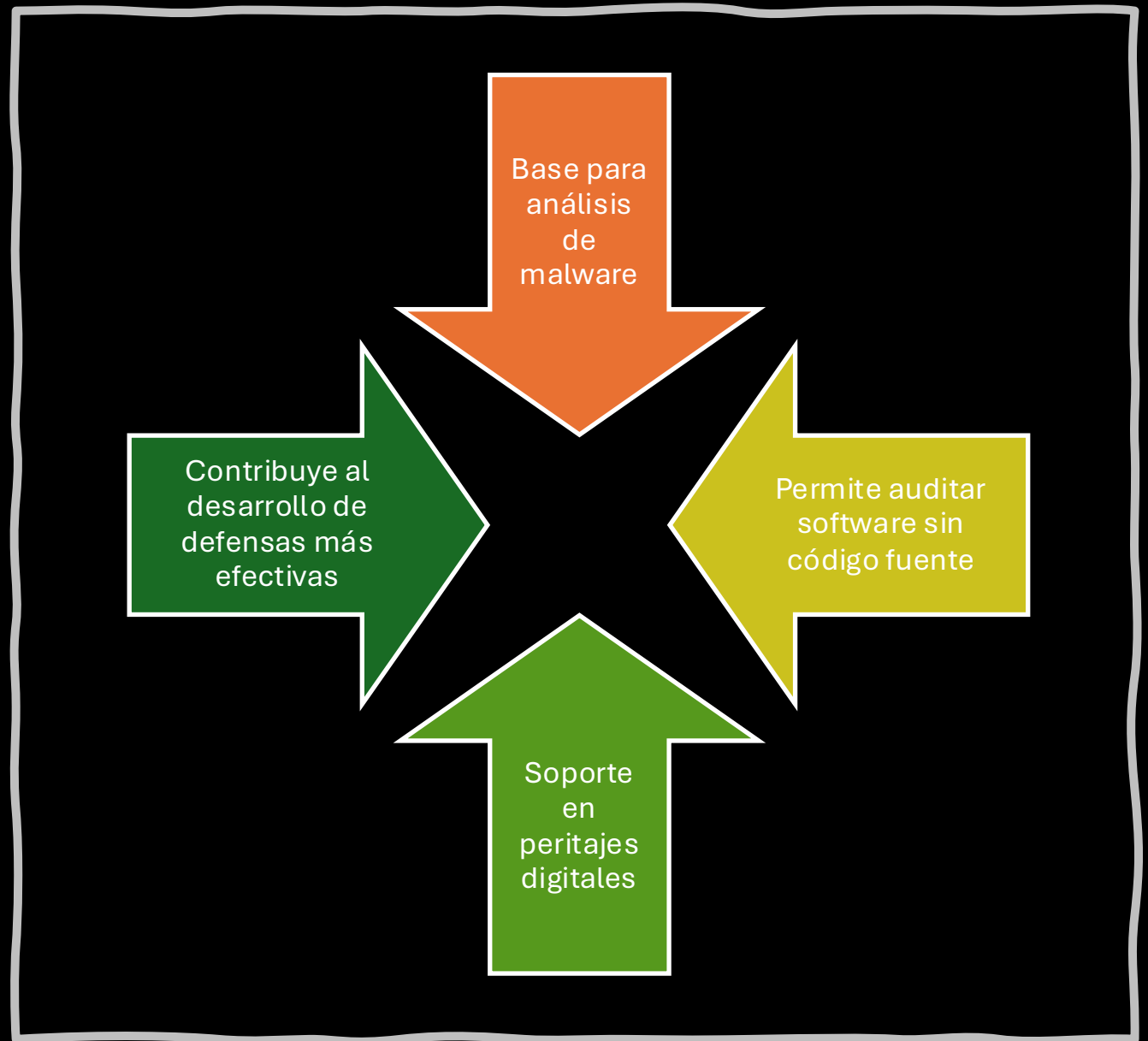


Años 90: auge de cracking y piratería

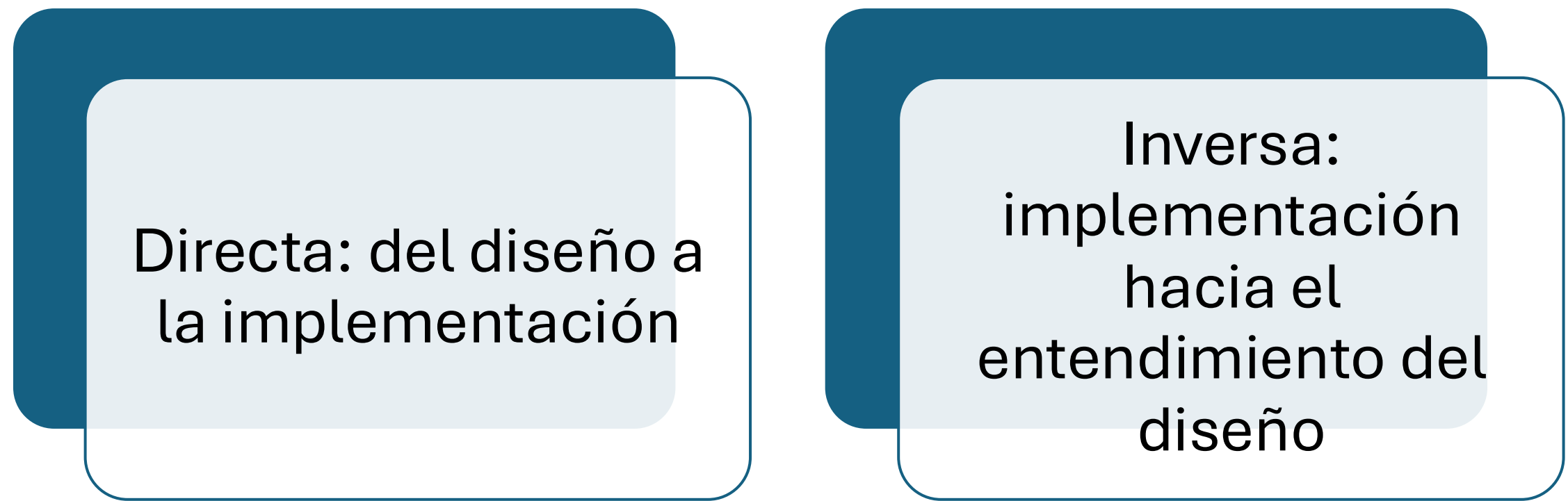


Actualidad: investigación avanzada de malware y seguridad

Importancia en la seguridad del software



Diferencia entre ingeniería directa e inversa



Directa: del diseño a la implementación

Inversa:
implementación
hacia el
entendimiento del
diseño

Herramientas Clave

Estático:

- **IDA Pro, Ghidra** (desensambladores/descompiladores).
- **Radare2** (framework CLI de análisis).
- **Binwalk** (análisis de binarios y firmware).

Dinámico:

- **OllyDbg, x64dbg** (debuggers).
- **Frida, Pin Tool** (instrumentación en tiempo real).
- **Cuckoo Sandbox** (entorno aislado de malware).

Flujo de Trabajo en Ingeniería Inversa

Recolección de información (hash, strings, dependencias).

Análisis estático inicial (estructura, secciones, funciones).

Ejecución controlada (debugging/sandboxing).

Documentación y diagramado del comportamiento.

Identificación de vulnerabilidades, backdoors o lógica crítica.

A large orange circle is positioned on the left side of the slide, partially cut off by the edge.

Casos de Uso Reales

Seguridad informática: Análisis de malware, exploits y rootkits.

Forense digital: Recuperación de funciones dañadas.

Auditoría de software:
Cumplimiento de licencias.

Compatibilidad: Adaptar sistemas heredados sin documentación.

Desafíos y Aspectos Éticos

Desafíos:

- Ofuscación, anti-debugging, packers.
- Alta curva de aprendizaje.

Ética y Legalidad:

- Puede estar restringida por licencias y leyes.
- En seguridad, se aplica bajo **consentimiento** y con fines educativos o defensivos.

Métodos de Reversión

Reversión Manual

Uso de desensambladores/debuggers.
Control total, pero más lento y requiere experiencia.

Reversión Semi-Automática

Herramientas como **Ghidra**, **IDA Pro** generan vistas gráficas.
Balance entre detalle y productividad.

Reversión Automática

Sandboxes y sistemas de análisis de malware.
Ideal para análisis inicial, pero menos preciso.

Desensamblado (Disassembly)



Convierte el binario en instrucciones en **Lenguaje Ensamblador**.



Permite ver:

Flujo de ejecución.
Llamadas a funciones/APIs.
Manipulación de memoria y registros.



Herramientas: **IDA Pro, Radare2, Hopper**.



Uso típico: comprender rutinas críticas, detectar instrucciones maliciosas.

Decompilado (Decompilation)

Convierte el binario a un **pseudo-código de alto nivel** (C, Java, etc.).

Ventajas:

- Más legible que el ensamblador.
- Facilita el análisis para quienes no dominan ASM.

Limitaciones:

- No siempre es perfecto (optimización del compilador, ofuscación).
- El código resultante puede perder estructura original.

Herramientas:
Ghidra, JADX
(Android), RetDec.

Ejemplos de Análisis Binario



1. Identificación de cabeceras y metadatos de un ejecutable

- **Ejemplo:** Abrir un archivo .exe en un editor hexadecimal (HxD, Hex Fiend, Bless).
- **Qué se analiza:**
 - Cabecera **PE (Portable Executable)** en Windows o **ELF (Executable and Linkable Format)** en Linux.
 - Dirección de entrada, secciones (text, data, rsrc).
 - Tamaño del archivo y alineación.
- **Uso en seguridad:** Permite ver si un malware está empaquetado, si tiene secciones inusuales o si esconde datos.

2. Extracción de cadenas de texto (strings)

- **Ejemplo:** Usar la herramienta strings sobre un binario.
- **Qué se encuentra:**
 - URLs sospechosas.
 - Direcciones IP.
 - Nombres de funciones o librerías.
- **Uso en seguridad:** Identificar rápidamente conexiones externas o funciones críticas (como llamadas a CreateProcess, WinExec, socket).

3. Desensamblado básico del código

- **Ejemplo:** Abrir el binario en IDA Pro o Ghidra.
- **Qué se analiza:**
 - Instrucciones en lenguaje ensamblador.
 - Saltos condicionales (jmp, call).
 - Funciones importadas del sistema operativo (como OpenFile, WriteProcessMemory).
- **Uso en seguridad:** Detectar rutinas maliciosas como keyloggers o exploits.

4. Comparación de binarios

- **Ejemplo:** Comparar dos versiones de un programa con BinDiff.
- **Qué se analiza:**
 - Diferencias en funciones entre versiones (útil para ver qué cambió un parche de seguridad).
- **Uso en seguridad:** Localizar vulnerabilidades corregidas o nuevas puertas traseras introducidas en una actualización.

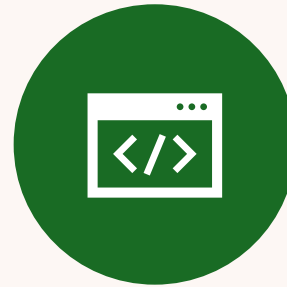
5. Detección de empaquetadores y ofuscadores

- **Ejemplo:** Analizar un binario con PEiD o Detect It Easy (DIE) en Windows.
- **Qué se analiza:**
 - Si el archivo fue comprimido con UPX, Themida u otro empaquetador.
- **Uso en seguridad:** Sospechar de malware que intenta esconder su código original.

El análisis binario se usa para:



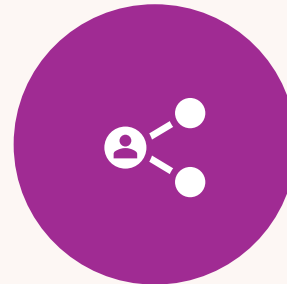
Revisar malware:
entender cómo se ejecuta
sin tener su código fuente.



Auditar software propietario: comprobar
que no tenga puertas
treras.



Comparar parches:
descubrir qué
vulnerabilidades se
corrigen.



Validar integridad:
asegurar que un programa
no fue alterado
maliciosamente.



Práctica: *Primer Análisis* *Binario*

Materiales necesarios

Un archivo binario simple

- Ejemplo: un .exe de Windows sin riesgo (puede ser el clásico calc.exe o notepad.exe).
- Alternativa multiplataforma: un ejecutable compilado en C “Hola mundo” (muy pequeño).

Herramientas gratuitas

- HxD (editor hexadecimal para Windows) o Hex Fiend (macOS).
- Comando strings (ya viene en Linux/macOS; en Windows se puede instalar con Sysinternals).

Cómo instalar y usar Sysinternals en Windows

Descargar Sysinternals Suite

Ve al sitio oficial:

<https://learn.microsoft.com/en-us/sysinternals/downloads/sysinternals-suite>

Descarga el archivo ZIP con todas las herramientas.

Extrae el contenido en una carpeta (ejemplo: C:\Sysinternals).

- 
- Sysinternals no requiere instalación, solo ejecutar.
Opciones:
 - **Ejecutar desde carpeta local:** entra a C:\Sysinternals y abre el .exe que necesites.
 - **Ejecutar en línea (sin descarga):**
 - Abre el **Explorador de Archivos**.
 - En la barra de direcciones escribe:
 - `\\live.sysinternals.com\tools`
-

Instrucciones paso a paso

- **Parte 1: Exploración con un editor hexadecimal**
- Abre el archivo ejecutable en HxD/Hex Fiend.
- Observa las primeras líneas (cabecera del archivo).
 - En Windows verán MZ (firma típica de ejecutables PE).
 - En Linux verán ELF.
- Que deben hacer:
 - Identifiquen las letras iniciales (MZ o ELF).
 - Busquen palabras legibles dentro del “ruido” binario.
- **Reflexión:** La cabecera indica el tipo de archivo y su formato, igual que una etiqueta de empaque.

Extracción de cadenas de texto con strings

- Ejecuta el comando en consola:
 - `strings nombre_del_archivo.exe | more`
- Identificar:
 - Nombres de librerías cargadas (KERNEL32.dll, USER32.dll).
 - Funciones interesantes (CreateFileA, WriteFile, socket).
 - Mensajes de error o texto embebido.
- **Reflexión:** Aunque no veamos el código fuente, las “pistas” de texto nos revelan funcionalidades internas.

Preguntas de análisis

01

¿Qué formato de archivo binario es? (PE, ELF, otro).

02

¿Qué funciones del sistema operativo parece usar?

03

¿Encontraron mensajes o URLs ocultas?

04

¿Por qué estas cadenas pueden ser útiles para identificar malware?

Análisis con código fuente

Solo posible cuando está disponible el código



Permite validar seguridad con mayor precisión

Análisis basado en comportamiento



Observar la interacción del
software con el sistema



Ideal para detectar malware

Comparación de enfoques

1

Binario: precisión
técnica

2

Comportamiento:
visión práctica

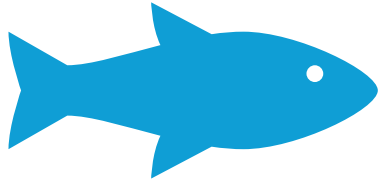
3

Código fuente:
mayor detalle, pero
menos frecuente

Métodos de Reversión



Inspección manual



Revisión directa del código
desensamblado



Requiere alta experiencia



Automatización
parcial

Scripts y plugins para
acelerar procesos

Apoyo en detección
de patrones

Patrones comunes en binarios maliciosos

Ofuscación de strings

Rutinas de cifrado

Llamadas sospechosas al sistema

Debugging y
trazas

Ejecución paso a
paso

Observación de
registros y
memoria

Métodos híbridos

Combinación de
análisis manual
y automático



Usados en
investigaciones
complejas

Ventajas y limitaciones

Ventajas:
descubrimiento
profundo de
amenazas

Limitaciones:
requiere tiempo y
experiencia

Decompilado y Desensamblado

¿Qué es el desensamblado?

Traducción del binario a lenguaje ensamblador

01100100
01101101

BINARIO



MOV AL, 0Ah
MOV BL, 0Bh

ENSAMBLADOR

Reconstrucción a pseudo-código
en alto nivel

¿Qué es el decompilado?

```
if (condition)
    executeAction();
}
```

Diferencias clave

- Desensamblado: bajo nivel, más preciso
- Decompilado: alto nivel, más entendible

DESENSAMBLADO

0110100

MOV AX, 01

ADD AX, 02

BAJO NIVEL,
MÁS PRECISO

DECOMPILADO

```
if (condition)
    executeAction();
}
```

ALTO NIVEL,
MÁS ENTENDIBLE

Análisis de Código

Análisis estático



ESTUDIAR EL SOFTWARE
SIN EJECUTARLO



IDENTIFICAR FUNCIONES,
LIBRERÍAS, PATRONES

Técnicas de análisis estático



FIRMAS DE MALWARE



ÁRBOL DE LLAMADAS



FLUJO DE DATOS

Herramientas de análisis estático

Ghidra

ClamAV
(detección de
firmas)

Checkmarx,
SonarQube

Análisis dinámico



OBSERVAR EL SOFTWARE
EN EJECUCIÓN



MONITOREO DE
PROCESOS Y RED

Técnicas de análisis dinámico

Sandboxing

Debuggers en
tiempo real

Monitoreo de
llamadas al
sistema

Herramientas dinámicas



CUCKOO SANDBOX



WIRESHARK



PROCESS MONITOR

Video 1

- <https://www.youtube.com/watch?v=KJvq2Scqa90>
- **¿Qué se observa?** Una lata o juguete que, al voltearse o moverla, emite un sonido de vaca (“Moooo”).
- **Analogía:** Como un software que produce una salida reconocible, pero con un mecanismo interno opaco.

Video 2

- <https://youtu.be/GdNJfbPnlQ8>
- **¿Qué se observa?** Un tubo que, al girarlo o moverlo, genera un sonido extraño tipo gemido (“groan”) causado por un mecanismo interior vibratorio.
- **Analogía:** Permite trabajar la noción de comportamiento observable sin acceso al diseño interno, similar al análisis dinámico del software

Video 3

- <https://youtu.be/4IS8cRhGA0g>
- **¿Qué se observa?** Un juguete (rattleback) que, al girarlo en una dirección, de repente se detiene y gira en sentido contrario sin intervención externa.
- **Analogía:** Un sistema con comportamiento “contraintuitivo” que muestra cómo configuraciones internas (asimetrías) pueden generar efectos inesperados — como ciertos tipos de ofuscación en software

Reflexion y analogía



CAJA MISTERIOSA = SOFTWARE BINARIO
(SIN CÓDIGO FUENTE)



COMPORTAMIENTOS CONTRAINTUITIVOS
(COMO LA RATTLEBACK) = LÓGICA OCULTA
O PATRONES NO OBVIOS EN MALWARE

Reverse Engineering Webinar de Corellium

- Si desean revisar la ingeniería reversa en aplicaciones moviles los invito a ver
- <https://www.corellium.com/reverse-engineering-webinar>
- En este webinar, los expertos de Corellium exploran cómo realizar ingeniería inversa en aplicaciones móviles para identificar vulnerabilidades en iOS y Android. Se abordan técnicas de análisis estático y dinámico, utilizando herramientas como Frida, y se demuestra cómo estas prácticas ayudan a mejorar la seguridad del software móvil